Yousef Yousef

Public medicine search and pharmacy directory frontend

Branch: Yousef | Plan begins when the team receives the starter.

Build two clear public screens from scratch, with simple components and agreed API shapes. No backend, database, sign-in or private inventory work is assigned to you.

Your code ownership

- apps/web/src/features/public-search/**

- apps/web/src/features/directory/**

Your agreed interfaces

- Create searchMedicines({q,area,onDutyNow}) using response fields agreed with Hamid; no request adapter is provided.

- Create listPublicPharmacies({area,onDutyNow}); public DTOs/types must be agreed and written by the team.

- Hamid owns these API endpoints. Read public fields only; cards never show stock counts/costs/batches.

- Use simple loading/error/badge components you create; ask Alaa for shared layout changes.

Start without waiting

Pull the foundation from main into your branch. Run npm ci, copy both apps/api/.env.example and apps/web/.env.example to .env on first setup, then npm run dev. The frontend shows only a plain startup message. No database models/migrations/seed or product features are implemented. Mustafa authors the database setup; follow his instructions after that PR merges.

Read the final specification, proposed API contract, acceptance checklist and your task folders. You build the assigned pages/services yourself. There are no prebuilt guards, calculations, forms, components, adapters or fixtures. Agree interfaces early and create your own small local test data to work independently.

Scope boundary

No backend, database, accounts, owner pages, maps or medical advice. Do not hard-code live results or hide an API error by silently showing sample data.

Week 1 - build your area

Days are relative, not the instructor deadline. Day 7 is a review/buffer day; adjust to your availability. Deliver small reviewed PRs throughout the week.

Day 1

Run the plain React foundation. Find your public-search/directory folders. Sketch two simple screens and create your first page/card component yourself. Agree public response fields with Hamid and routes with Alaa.

Day 2

Add a labeled medicine input and Search button to PublicSearchPage. Save the submitted query in component state; avoid sending a request on every keypress.

Day 3

Add area text input and on-duty checkbox. Send q/area/onDutyNow through a search request function you create in your feature. Prevent empty medicine submission and keep entered values visible.

Day 4

Build PharmacyCard yourself: name, area, address, phone link, on-duty badge and current/next duty time. Create a small local fixture file to test cards; format dates and handle missing inventory timestamps clearly.

Day 5

Implement loading, no-results and error states yourself, with a helpful message and Retry button. Exercise each state with your own local fixtures or simulated requests; no shared state component is prebuilt.

Day 6

Add area/on-duty filters to DirectoryPage using listPublicPharmacies. Reuse PharmacyCard. Directory opens without requiring a medicine query or sign-in.

Day 7

Buffer/review: check phone-sized layout and keyboard labels. Open Week 1 UI PR with screenshots for search, directory and empty/error examples.

Week 2 - integrate and prove

Day 8: API integration. Day 12: core acceptance. Days 13-14: English rehearsal and fixes. No new optional features while CORE is failing.

Day 8

With Hamid, switch to real API integration and test search against seeded data. Mock mode remains for isolated styling, but final checks must use real API responses.

Day 9

Verify partial/case-insensitive scientific-name search, an area with no matches and area+on-duty together. Report API mistakes to Hamid instead of calculating stock/status in React.

Day 10

Check request changes do not show stale results. Add a visible result count and clear/reset filters; retry errors without losing the current query.

Day 11

Test directory current/next/null duty examples and missing timestamp. Add accurate labels so 'off duty' does not claim a pharmacy is closed; duty status is all we know.

Day 12

Check accessibility: input labels, keyboard order, buttons, meaningful status text and responsive cards. With Hamid, check your pages in dark mode/Arabic if those optional controls are ready.

Day 13

Follow the citizen demo: search medicine, apply duty filter, inspect address/phone, try a no-match term. Confirm no private details appear anywhere in your cards.

Day 14

Buffer/final rehearsal. Present the citizen search and directory in English, show empty/error behavior and explain what information citizens can and cannot see.

Ready-to-merge checklist

[ ] Search requires a nonempty medicine query and sends the correct filters; directory does not require medicine text.

[ ] All cards show only allowed public fields; no quantity/cost/barcode/batch/subscription information.

[ ] Loading, zero results, backend errors, retry and result count work without reloading the browser.

[ ] Current/next duty and null values are clear; on-duty badge does not falsely claim general opening hours.

[ ] Real API-mode search, scientific names, combined filters, keyboard and narrow-screen layout tested.

Handoffs and dependencies

- Start immediately with your own page sketches/components and small local fixtures. Week 1 frontend work does not need a running backend.

- Ask Alaa for shared component/style help; keep changes inside your feature folders.

- Day 6-7: get Hamid's public JSON examples; Day 8 integrate real API integration.

- Report backend defects with search term, filters and observed result so Hamid can reproduce.

Every pull request

Commit only your intended files; push your branch; open a PR into main. Include the task/rule IDs, a short explanation, and tests or screenshots. Run npm run typecheck, npm test and npm run build. Backend contributors also document their endpoints in Swagger. Alaa reviews/merges; keep .env, node_modules, dist and temporary files untracked.

Your English presentation segment

Demonstrate a medicine search, area and duty filtering, the pharmacy contact card and a no-results state. Explain that public availability does not expose exact quantities.

Everyone should understand the whole flow: owner onboarding -> pharmacist stock -> dashboard -> public search. Be ready to explain a rule outside your own feature. Final acceptance uses real API requests and persisted PostgreSQL data; locally authored test fixtures are only for development.